QQUANTT.COM

by mehul.engineer

Bit-Based Randomized Entropy System
Scheduler-Based RNG

Mehul Singh

ID: sys-eng/001

March 2026-March 2026

Systems Engineering

OVERVIEW

BBRES-RNG (Bit-Based Randomized Entropy System - Scheduler-Based RNG) is a multithreaded random number generator that harvests real-time entropy from OS thread-scheduling chaos. Instead of relying on standard library algorithms or fixed mathematical seeds, it deliberately creates controlled concurrency and treats the unpredictable timing behaviour of the OS thread scheduler as its entropy source. Every number it produces is shaped by the live, non-deterministic state of the host machine, making each output practically unrepeatable.

This departs from traditional PRNG design in several respects: traditional PRNGs are seed-based and deterministic from the same seed, rely on mathematical formulas (LCG, Mersenne Twister, etc.), are reproducible given the same initial state, and execute single-threaded. BBRES-RNG is instead entropy-based and non-deterministic by design, harvests timing noise from OS thread scheduling, is practically unreproducible because it is tied to real-time system state, and is built on a multi-threaded concurrency architecture.

DESIGN & PIPELINE ARCHITECTURE

The generation pipeline proceeds through three stages.

Stage 1 - Thread Spawning & Controlled Race Conditions: multiple worker threads (`randomBitGenRNG`) launch simultaneously; the OS scheduler decides their execution order, which is inherently unpredictable and varies by microseconds depending on CPU state, system load, and kernel-level scheduling.

Stage 2 - Timing-Based Entropy Collection: each worker thread races to record its id into a shared flag array (guarded by a synchronized `getBit` method), and the arrival order becomes the raw entropy signal; a separate `AtomicIntegerArray` is used to coordinate readiness across thread groups without a global lock.

Stage 3 - Bitwise Mixing & Aggregation: collected arrival-order values from the first and last ~20% of finishers are XOR-combined, then passed through a small xorshift-style mixing step (`xor ^= xor << 10; xor ^= xor >>> 23; xor ^= xor << 7;`) to produce one random bit per cycle; bits are assembled by `RNG.java` into arbitrary-range integers.

The orchestration layer (`randomBitGeneratorModifiedRoot.java`) segments the requested concurrency level `n` into blocks of up to 32 threads (capped at 64 segments), runs each block through `modRandomBitGenRNG` , and XORs the per-block results together to form the final bit. Two generation techniques are exposed:

G1 (`generateRandBitG1`), which segments threads in their natural id order

G2 (`generateRandBitG2`), which first performs a Fisher-Yates-style shuffle of thread ids (itself driven by a nested BBRES-RNG v1 call) before segmenting, adding an extra layer of unpredictability to the scheduling races.

The public API, `RNG.java` , accepts a `(min, max)` range together with an optional concurrency level `n` (valid range 3-1000, default 71) and a technique selector `randTech` (1 or 2, default 1). It repeatedly requests random bits equal to the bit-length of the range, reconstructs an integer, masks off the sign bit, and rejects out-of-range draws (rejection sampling) until a valid value in `[min, max]` is produced.

STATISTICAL VALIDATION METHODOLOGY

BBRES-RNG was validated against a comprehensive 45-test battery spanning NIST SP 800-22 core and extended tests (including Binary Matrix Rank, Non-Overlapping Template, Overlapping Template, and Linear Complexity), distribution/uniformity tests, spectral analysis (FFT, compression ratio, binary derivative, turning-point test), entropy measurements, autocorrelation profiling, pattern detection, adversarial machine-learning attacks (Logistic Regression, Gradient Boosted Trees, MLP Neural Network), a cryptographic wrapper validation (SHA-256 CSPRNG), and integer-level distribution and sequence tests (Anderson-Darling, collision, maximum-of-t, Spearman correlation, median, and others).

Test parameters: bit sample size 10,918,505 bits; integer sample size 100,000 integers over `[0, 999]` ; 45 tests per RNG; significance level α = 0.01. The same battery was run head-to-head against Java's `SecureRandom` and `Math.random()` for direct comparison.

RESULTS

Scorecard: BBRES-RNG passed 45/45 tests (architecture accepted, no failures). Java `SecureRandom` also passed 45/45. Java `Math.random()` passed 43/45, failing Maurer's Universal Statistical test and the Gap Test (KS) - both of which specifically probe for the kind of subtle periodic/compressible structure and non-uniform gap distribution that linear-congruential-style generators can exhibit.

NIST SP 800-22 core suite (9/9 passed): Frequency/Monobit (p=0.441897), Block Frequency M=128 (p=0.143304), Runs Test (p=0.670559), Longest Run of Ones (p=0.046389), Cumulative Sums forward/reverse (p=0.164091 / p=0.656919), Approximate Entropy m=2 (p=0.851301), and Serial m=2 delta1/delta2 (p=0.679895 / p=0.671131).

NIST SP 800-22 extended suite (7/7 passed): Maurer's Universal Statistical (p=0.722786), Poker Test m=4 (p=0.517531), Random Excursion Variant (p=0.933697), Binary Matrix Rank (p=0.982609), Non-Overlapping Template m=9 (p=0.361738), Overlapping Template m=9 (p=1.000000), and Linear Complexity (p=0.620280).

Distribution/uniformity (3/3), spectral/structural (4/4), and adversarial/cryptographic tests (6/6) all passed, including three independent ML attack models - Logistic Regression, Gradient Boosted Trees, and an MLP neural network - each of which only reached ~50% prediction accuracy on held-out bits, equivalent to random guessing. A SHA-256-based CSPRNG wrapper seeded with BBRES output was re-tested for predictability and also passed (p=1.000000).

Entropy quality: Shannon entropy (8-bit) of 7.999860 out of a theoretical maximum of 8.000000 (99.998% of maximum); min-entropy of 7.944862; transition rate of 0.500064 versus an ideal of 0.500000; and a bit balance of 49.988:50.012 (1s:0s) versus an ideal 50:50. Autocorrelation across lags of 1, 2, 4, 8, 16, 32, 64, 128, and 256 bits remained within ±0.0005 of zero at every lag, indicating no detectable serial dependence.

Integer-level tests (16/16 passed): covering Chi-Square Uniformity, Mean (Z-test), Variance (Chi-sq), Binned Goodness-of-Fit, Birthday Spacing, Coupon Collector, Collision Test, Anderson-Darling, Skewness/Kurtosis, Maximum-of-5, Runs Up/Down, Lag-1 Autocorrelation, Gap Test (KS), Permutation (t=5), Spearman Rank Correlation, and Median Test.

API & USAGE

The public interface is a single class, `bbresRNG.RNG` , with four overloads:

`bbresRNG()` - random number in [0, 10] with default concurrency (n=71) and default technique (G1).
`bbresRNG(min, max)` - random number in [min, max] with default concurrency and technique.
`bbresRNG(min, max, n)` - random number in [min, max] with n concurrent worker threads (valid range 3-1000; invalid values fall back to the default of 71).
`bbresRNG(min, max, n, randTech)` - full control over range, concurrency, and generation technique (1 = G1, 2 = G2/shuffle-based; invalid values fall back to technique 1).

Example usage:

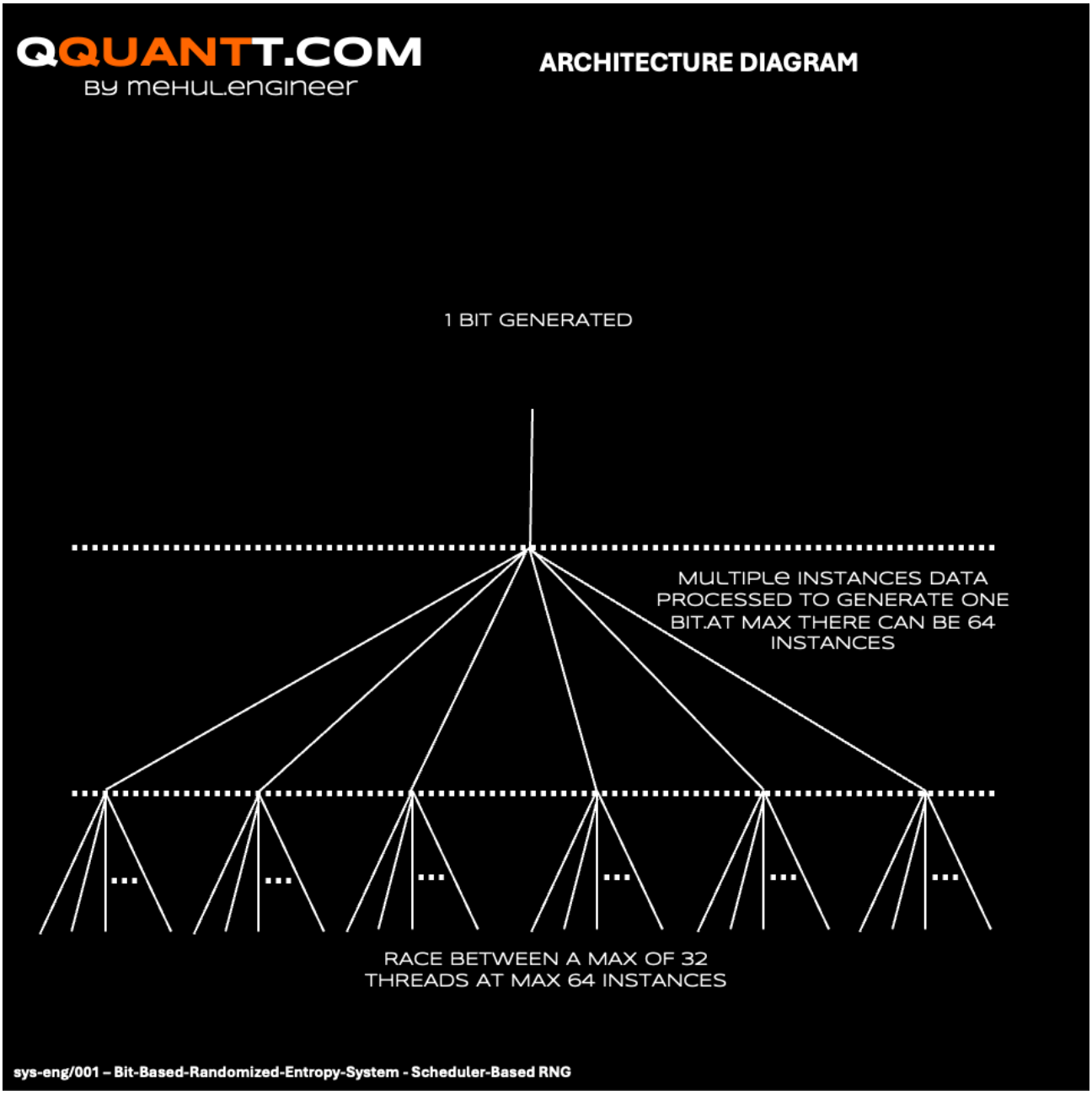
```
RNG rng = new RNG();
System.out.println(rng.bbresRNG()); // [0, 10]
System.out.println(rng.bbresRNG(5, 15)); // [5, 15]
System.out.println(rng.bbresRNG(1, 100, 50)); // n=50 worker threads
System.out.println(rng.bbresRNG(0, 10000, 35, 2)); // G2, n=35
```

Higher values of `n` increase entropy at the cost of additional computation time; JDK 8+ is required to build and run the project (`javac src/Main.java` then `java -cp src Main`).

REPOSITORY STRUCTURE & LICENSING

The project is organized as `src/Main.java` (entry point and usage examples) and `src/bbresRNG/` , containing `RNG.java` (public API), `randomBitGeneratorModifiedRoot.java` (core bit generator and thread orchestration, G1/G2), `modRandomBitGenRNG.java` (worker-thread variant collecting timing data in parallel), and `randomBitGenRNG.java` (base worker thread launched for entropy harvesting). A `docs/` directory holds the architecture documentation and the full validation reports (BBRES-RNG, Java SecureRandom, and Java Math.random() combined reports) referenced in the Results section above.

SUPPLEMENTARY MATERIAL



BBRES-RNG Pipeline Architecture

View full size

BBRES-RNG Combined Validation Report: https://qquantt.com/files/sys-eng/001/docs/Reports/BBRES-RNG%20Combined%20V

Java SecureRandom Combined Validation Report: https://qquantt.com/files/sys-eng/001/docs/Reports/java.security.SecureRand

Java Math.random() Combined Validation Report: https://qquantt.com/files/sys-eng/001/docs/Reports/Java%20Math.random()%2

RNG Comparison Report: https://qquantt.com/files/sys-eng/001/docs/Reports/Combined Comparision Report/Combined Compar

REFERENCES

Download .zip: files/sys-eng/001/BBRES-RNG.zip

Repository: https://github.com/mehul-engineer/Bit-Based-Randomized-Entropy-System-Scheduler-Based-RNG.git

Collaborate: mailto:research@qquantt.com